

VULNERABILITY ASSESSMENT REPORT

Cyber Security Task 1 — Read-Only Security Audit of a Live Web Target

Target Platform: testphp.vulnweb.com (public test environment)

Internship Batch: Cyber Security Cohort 2026

Assessment Date: 29 June – 2 July 2026

Prepared For: Future Interns Training Programme

Document Status: Final Deliverable

Document Control & Ethical Attestation

This document is the operational deliverable for Cyber Security Task 1 of the Future Interns programme. In line with the ethical rules set out in the task brief, the engagement was carried out under a strictly **read-only, non-intrusive scope**.

No login bypass attempts, exploitation, brute-force attacks, or denial-of-service activity were performed against the target. All analysis was passive or low-impact: HTTP response header review, public service/port mapping, and browser-based configuration inspection.

Assessment Parameters

Parameter	Detail
Testing modality	Passive header review, public port/service mapping, browser-based configuration checks.
Tools employed	OWASP ZAP v2.17.0 (passive scan), Nmap 7.99, Browser Developer Tools.
In scope	Public-facing pages, response headers, cookie attributes, open service enumeration.
Out of scope	Database injection, credential/brute-force attacks, exploitation of application logic, denial-of-service testing.

Table of Contents

1. Executive Summary & Risk Matrix	3
2. Methodology & Read-Only Testing Framework	4
3. Network Enumeration Log (Nmap)	5
4. Passive Application Traffic Telemetry	6
5. Findings & Plain-Language Breakdown	7
5.1 Content Security Policy (CSP) Header Omission	7
5.2 Missing Anti-Clickjacking Protection	8
5.3 Information Disclosure via Server Banner	9
5.4 Insecure SameSite Cookie Attribute	9
5.5 Missing Content-Type Sniffing Protection	10
5.6 Data Leakage on Unauthenticated Redirects	10
6. Hardening Configuration Blueprint	11
7. Remediation Roadmap	12

1 Executive Summary & Risk Matrix

Every business with a public website carries some exposure, even when no critical exploit exists. Clients rarely ask for a penetration test — they ask three simple questions: *Is our site safe? What are the immediate risks? What should be fixed first?* This report answers those questions for the assessed target using a read-only methodology.

The assessment found **no high-severity or system-level vulnerabilities** during the baseline scan. All identified issues are **configuration-level weaknesses** — missing or misconfigured HTTP security headers and cookie attributes — rather than flaws in application logic. These are common, low-effort-to-fix issues that nonetheless widen the attack surface for client-side attacks such as cross-site scripting and clickjacking.

Consolidated Risk Matrix

Finding	Risk	CWE	Business Impact
Missing Content Security Policy (CSP)	MEDIUM	CWE-693	Allows the browser to run unauthorised scripts if injected — the core enabler of XSS attacks.
Missing anti-clickjacking header	MEDIUM	CWE-1027	Site can be embedded in a hidden frame, tricking users into clicking things they cannot see.
Server banner information disclosure	LOW	CWE-497	Exposes exact server/software version, making it easier for an attacker to target known exploits.
Insecure cookie SameSite attribute	LOW	CWE-1275	Secondary cookie can be attached to cross-site requests, raising CSRF exposure.
Missing X-Content-Type-Options header	LOW	CWE-693	Browser may guess a file's type and execute disguised malicious content.
Data exposure on unauthenticated redirect	LOW	CWE-201	Page content is sent in the response body before the redirect completes.

2 Methodology & Read-Only Testing Framework

The assessment followed a linear, low-impact process designed to evaluate configuration posture without any risk of disrupting the live target.

Passive Inspection → Network Mapping → Header Analysis → Cookie/Storage Audit →
Remediation Matrix

Testing Sequence

- 1. Passive inspection.** Reviewed baseline response headers and rendered pages during normal browsing, without submitting any crafted or malicious input.
- 2. Network interface mapping.** Ran low-impact Nmap scans to confirm reachable services and open ports on the target host.
- 3. Header configuration analysis.** Inspected HTTP response headers for security controls such as CSP, X-Frame-Options, and X-Content-Type-Options.
- 4. Cookie and storage auditing.** Used browser developer tools to check the Secure, HttpOnly, and SameSite attributes on session cookies.

This methodology intentionally excludes anything that could alter server state, degrade availability, or access data belonging to another user — consistent with the ethical scope defined for this task.

3 Network Enumeration Log (Nmap)

Initial reconnaissance used Nmap to confirm connectivity and identify the general firewall posture of the target, **testphp.vulnweb.com**, a public test environment used for practising non-destructive web security assessment.

Baseline connection scan

```
$ nmap testphp.vulnweb.com
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-29 20:15 +0530
Nmap scan report for testphp.vulnweb.com (44.228.249.3)
Host is up (0.0072s latency).
Not shown: 1000 filtered tcp ports (no-response)
Nmap done: 1 IP address scanned in 52.82 seconds
```

Service-detection scan, bypassing ICMP filtering

```
$ nmap -Pn -sV testphp.vulnweb.com
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-29 20:21 +0530
Nmap scan report for testphp.vulnweb.com (44.228.249.3)
Host is up.
Not shown: 1000 filtered tcp ports (no-response)
Service detection performed. Scan done in 201.62 seconds
```

The 201.62-second runtime indicates the firewall silently drops unsolicited SYN packets rather than returning an immediate RST, which forces Nmap into a slower retransmission window. This is a defensive configuration, not a vulnerability in itself.

4 Passive Application Traffic Telemetry

The OWASP ZAP passive-scan spider crawled the target and catalogued **16 reachable endpoints** across its public page tree, without submitting any authentication credentials or attack payloads.

HTTP Status Code Distribution

Status class	Share	What it reflects
2xx — Success	69%	Normal page rendering and static asset delivery.
3xx — Redirection	30%	Routing and access-control gates redirecting unauthenticated requests.
4xx / 5xx — Error	1%	Requests to invalid or missing paths.

Asset Type Breakdown

Asset type	Share	Typical use
text/html	43%	Core page layouts and forms.
image/png	25%	Graphics, logos, product images.
text/css	12%	Styling and layout sheets.
text/javascript	12%	Client-side scripting and interactivity.
image/vnd.microsoft.icon	6%	Browser favicon.

Method breakdown: **87%** of observed traffic used **GET** requests, while **12%** used **POST** for state-changing actions such as login and configuration submissions (no credentials were submitted during testing).

5 Findings & Plain-Language Breakdown

5.1 Content Security Policy (CSP) Header Omission

MEDIUM

CWE: CWE-693 | Plugin ID: 10038

Plain-language explanation

A Content Security Policy works like an ID check at the door — it tells the browser exactly which sources are allowed to run scripts on the page. Without it, the browser will execute any script that ends up on the page, which is the core mechanism behind Cross-Site Scripting (XSS) attacks.

Evidence observed on target

- <http://testphp.vulnweb.com/> — response headers contain no Content-Security-Policy directive.
- <http://testphp.vulnweb.com/login.php> — same omission observed on the authentication page.
- <http://testphp.vulnweb.com/search.php?> — same omission observed on the search endpoint.

Recommended fix

Add a baseline Content-Security-Policy header to all outgoing HTML responses:

```
Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline';
```


5.2 Missing Anti-Clickjacking Protection

MEDIUM

CWE: CWE-1027 | Plugin ID: 10038

Plain-language explanation

Without a framing restriction, an attacker can load your site inside an invisible frame on their own page. Visitors believe they are clicking the attacker's page, while their clicks are actually being delivered to hidden buttons on your site underneath — for example a hidden "change password" button.

Evidence observed on target

- <http://testphp.vulnweb.com/> — no X-Frame-Options or frame-ancestors directive present.
- Confirmed by successfully loading the target inside a local test <iframe> during passive review.

Recommended fix

Add the standard anti-framing header to server responses:

```
X-Frame-Options: SAMEORIGIN
```

5.3 Information Disclosure via Server Banner

LOW

CWE: CWE-497 | Plugin ID: 10036

Plain-language explanation

The server openly reports its exact software version in every response. Knowing the precise version shortens an attacker's reconnaissance time, since they can go straight to checking known issues for that specific release instead of probing blindly.

Evidence observed on target

- Response header observed: Server: nginx/1.19.0

Recommended fix

Suppress verbose version strings at the web server level, for example in nginx:

```
server_tokens off;
```

5.4 Insecure SameSite Cookie Attribute

LOW

CWE: CWE-1275 | Plugin ID: 10054

Plain-language explanation

Session cookies control who is "logged in" as whom. The primary session cookie is correctly restricted, but a secondary configuration cookie lacks a same-site restriction — meaning it can be attached to requests that originate from another website, a building block of Cross-Site Request Forgery (CSRF) attacks.

Evidence observed on target

- Set-Cookie: PHPSESSID=...; Path=/ (missing SameSite attribute)
- Set-Cookie: login=1; Path=/ (missing Secure and SameSite attributes)

Recommended fix

Explicitly set Secure, HttpOnly, and SameSite on all session and state cookies:

```
setcookie("PHPSESSID", $value, [  
    "secure" => true,  
    "httponly" => true,  
    "samesite" => "Strict"  
]);
```

5.5 Missing Content-Type Sniffing Protection

LOW

CWE: CWE-693 | Plugin ID: 10021

Plain-language explanation

Servers are supposed to tell the browser exactly what kind of file it is sending. Without a no-sniff instruction, some browsers will inspect the file content and guess its type instead — which an attacker can abuse by disguising an executable script as a harmless image or text file.

Evidence observed on target

- All tested responses (text/html, image/png, text/javascript) lacked the X-Content-Type-Options header.

Recommended fix

Force browsers to respect the server-declared content type:

```
X-Content-Type-Options: nosniff
```

5.6 Data Leakage on Unauthenticated Redirect

LOW

CWE: CWE-201 | Plugin ID: 10044

Plain-language explanation

When a logged-out visitor requests a restricted page, the server correctly redirects them to the login page — but it still sends a small amount of the restricted page's HTML in the response body before the redirect fires, leaking internal page structure to anyone who inspects the raw response.

Evidence observed on target

- GET /admin/ (unauthenticated) — HTTP 302 redirect to /login.php, with 346 bytes of page markup included in the response body.

Recommended fix

Terminate script execution immediately after issuing the redirect header:

```
header("Location: /login.php");  
exit(); // stop further output
```

6 Hardening Configuration Blueprint

The individual fixes above can be deployed together as a single drop-in configuration. The blocks below cover the two most common web server platforms.

Apache (.htaccess or virtual host config)

```
Header set X-Content-Type-Options "nosniff" always
Header set X-Frame-Options "SAMEORIGIN" always
Header set Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:;" always

ServerTokens ProductOnly
ServerSignature Off
```

nginx (server block)

```
server_tokens off;

add_header X-Content-Type-Options "nosniff" always;
add_header X-Frame-Options "SAMEORIGIN" always;
add_header Content-Security-Policy "default-src 'self'; script-src 'self';" always;
```

7 Remediation Roadmap

A phased schedule keeps remediation manageable without disrupting live operations.

Phase	Timing	Action
Phase 1	Days 1–2	Deploy global response headers: X-Frame-Options, X-Content-Type-Options, and a baseline CSP, across the primary web server configuration.
Phase 2	Days 3–5	Update cookie setters to enforce Secure, HttpOnly, and SameSite=Strict on all session and configuration cookies. Add exit() immediately after redirect headers to stop residual data leakage.
Phase 3	Ongoing	Disable server version banners (ServerTokens ProductOnly / server_tokens off). Re-run OWASP ZAP passive scans periodically to catch configuration regressions.

Closing note. No high-severity or system-level vulnerabilities were identified. All findings are configuration-level and can be resolved with the header and cookie changes above, typically within one working week, with no code-level refactoring required.

— End of Report — Future Interns Cyber Security Task 1 Deliverable —